

When FLOPs Mislead: Benchmarking Neural Network Inference Energy on Apple Silicon

Aryan Shah

Texas Academy of Mathematics and Science
University of North Texas
Denton, TX, USA
aryanshah.2431@gmail.com

Abstract—Energy is a deployment constraint for edge-AI systems, but neural-network efficiency is still often discussed using architecture-level proxies such as FLOPs, parameter count, and model size. We first reproduce a five-model Apple-Silicon CPU-only PyTorch comparison, then add a new direct powermetrics audit over 17 architecture variants on an Apple M4 Pro. The new audit contains 170 measured windows, 30 seconds per window, raw power logs, per-window inference counts, confidence intervals, and full environment metadata. Three follow-up audits then test the result under MPS/Metal execution, CPU batch size 4, and a smaller 128×128 CPU input. Across the 17 directly measured CPU batch-1 architectures, latency explains nearly all between-architecture energy variation ($R^2 = 0.9925$), while FLOPs is much weaker ($R^2 = 0.4008$) and parameter count is weaker still ($R^2 = 0.0977$). The expanded data preserve the central five-model result: runtime behavior, not nominal arithmetic count, determines energy ordering under this fixed CPU backend, and the sensitivity checks show which parts of the ranking persist when the backend, batch size, or input size changes. For example, ConvNeXt-micro has only 0.448 GFLOPs but consumes slightly more energy than a 3.414-GFLOP residual CNN because both run at roughly the same latency. A residualized conditional WGAN-GP is retained only as supplemental sparse-trial modeling. The results support a practical benchmarking rule for edge inference: report direct latency and auditable power traces under the deployment runtime instead of ranking models by FLOP count alone.

Index Terms—edge AI, benchmarking, neural network inference, energy measurement, Apple Silicon, energy-delay product

I. Introduction

High-performance embedded and edge systems increasingly run neural-network inference under strict energy and latency budgets. Model selection is often guided by FLOPs, parameter count, or model size because these quantities are easy to compute before deployment. They are useful descriptors, but they do not directly measure wall-clock work, memory traffic, kernel efficiency, or the behavior of a specific runtime on a specific processor [1]–[3].

This paper studies a narrow but practical question: for a fixed Apple Silicon CPU-only PyTorch runtime, which common architecture proxy best tracks measured inference energy? The answer in this benchmark is latency, not FLOPs. This is important for HPEC-style edge deployments because the relevant cost is the energy actually

paid by the hardware under the deployment runtime, not the nominal arithmetic count of the neural network.

The contributions are:

- A five-architecture Apple-Silicon inference-energy benchmark comparing MobileNetV3-Small, MobileNetV2, ResNet-18, Tiny-ViT-5M, and EfficientNet-B0.
- A new 17-architecture Apple M4 Pro direct-energy audit with 170 powermetrics windows, raw logs, per-window inference counts, confidence intervals, and environment metadata.
- Three targeted follow-up audits for limitations: MPS/Metal backend comparison, CPU batch-size sensitivity at batch 4, and CPU input-size sensitivity at 128×128 .
- A compact analysis showing that latency is a much stronger predictor of energy than FLOPs in both the original five-model table and the expanded 17-architecture direct-energy table.
- A scoped synthetic-trial modeling note: residualized conditional GANs can support exploratory repeated-trial analysis, but they should not replace independent power measurements.

II. Related Work

Efficient neural-network design has produced architectures such as MobileNet, MobileNetV2, MobileNetV3, EfficientNet, SqueezeNet, and ShuffleNet, which reduce arithmetic through depthwise separable convolutions, inverted residuals, compound scaling, and channel grouping [3]–[8]. These designs often improve accuracy/FLOP tradeoffs, but hardware energy depends on more than FLOPs.

Prior systems work shows that data movement and memory hierarchy behavior can dominate arithmetic energy [1], [2]. Hardware-aware model design similarly emphasizes that latency and hardware utilization can diverge from FLOP counts [3], [6]. Green AI and ML systems benchmarking motivate reporting measured deployment costs rather than abstract complexity alone [9], [10]. This paper adds a small, auditable Apple-Silicon case study centered on per-inference energy.

III. Methodology

A. Runtime and Hardware Scope

All primary results use CPU-only PyTorch inference with batch size 1 and input shape $1 \times 3 \times 224 \times 224$. CPU-only execution was chosen to isolate a portable runtime path and avoid Neural Engine or Metal scheduling effects. It does not claim to be the most energy-efficient Apple Silicon backend.

The repository contains multiple Apple-Silicon energy sources. The first is the paper-aligned original five-model mean table, which reports one mean latency and one mean energy per model from Apple-Silicon CPU inference measured with powermetrics at 1 Hz. The raw logs for that original table are unavailable, and the local artifact does not identify it as M1, so we do not infer confidence intervals or a specific chip generation for it.

The main new evidence is a 17-architecture Apple M4 Pro audit collected in measured_energy_powermetrics_m4pro_arch17_cpu. It uses 10 30-second windows per architecture, 170 measured windows total, 1 Hz powermetrics sampling, 20 warmup inferences before each window, randomized architecture order, one CPU thread, batch size 1, and input shape $1 \times 3 \times 224 \times 224$. The audit preserves raw powermetrics text logs, per-window inference counts, per-window energy estimates, summary statistics, confidence intervals, and a JSON environment record. Earlier five-model M4 and retained M5 summaries are treated as supporting audits, not merged with the new 17-architecture table.

After the main audit, three shorter follow-up audits were collected on the same M4 Pro environment to address backend and input-scope limitations. The first repeats the 17-architecture suite with PyTorch MPS/Metal at batch size 1 and 224×224 input. The second repeats the CPU suite at batch size 4 and 224×224 input. The third repeats the CPU suite at 128×128 input for 15 architectures; the two patch-mixer variants are excluded from this input-size check because their token MLPs are shape-fixed to the 224×224 registry definition. These follow-up audits use five 20-second windows per architecture and are reported separately from the main CPU batch-1 table.

B. Metrics

We report per-inference energy E in joules, latency L in milliseconds, and average power $P = 1000E/L$. Because power is sampled once per second, energy is interpreted as window-level energy divided by the number of inferences completed in the window. We also report Energy-Delay Product (EDP),

$$\text{EDP} = E \cdot L. \quad (1)$$

EDP is not accuracy-normalized; it measures energy-latency cost for fixed input-shape inference.

C. Benchmark Procedure

Each model is instantiated once, switched to evaluation mode, and run with gradients disabled. Warmup inferences are discarded so that module loading and one-time setup do not dominate short windows. During each measured window, the script repeatedly executes forward passes until the window expires, records the number of completed inferences, and divides the package-energy estimate for that same window by the inference count. Raw power-metrics output is retained, so the summary CSV is not the only source of evidence.

This windowed procedure is deliberately conservative for sub-100 ms inference. It does not try to attribute a single instantaneous power sample to one forward pass. Instead, it measures many repeated inferences under a controlled loop and reports the average energy paid by that loop.

D. Supplemental Synthetic Modeling

The repository includes a conditional WGAN-GP pipeline used to explore synthetic repeated trials under sparse measurement budgets. An initial multi-device raw-scale model mode-collapsed. The revised model uses within-combination residuals:

$$\tilde{x} = \text{clip}((x - \mu_c)/\sigma_c, -3, 3). \quad (2)$$

Generated residuals are mapped back using the same μ_c and σ_c . This synthetic component is treated as supplemental. It can reproduce grounded means and rankings, but its spreads are not a substitute for held-out measured trial-level energy data.

IV. Results

Table II summarizes the five-model paper-aligned original mean table. The key result is that energy follows latency rather than FLOPs. EfficientNet-B0 has only 0.39 GFLOPs, but it has the largest latency and energy. ResNet-18 has 1.82 GFLOPs, yet consumes less than half the energy of EfficientNet-B0 in this setup.

A. Predictor Analysis

We fit univariate linear models predicting energy from latency, FLOPs, and parameter count in the original mean table. Latency explains nearly all between-model energy variation ($R^2 = 0.99999$), while FLOPs explains almost none ($R^2 = 0.00008$). Leave-one-out checks preserve the same qualitative result: latency R^2 remains above 0.99997, while FLOPs remains weak.

B. Seventeen-Architecture M4 Pro Energy Audit

Table III reports the new direct M4 Pro audit. Each architecture has 10 measured 30-second windows, with raw logs and per-window rows stored in the artifact directory. Confidence intervals are Student- t 95% intervals over the 10 measured windows. This table replaces the earlier latency-only 17-architecture sweep as the main expanded

TABLE I
Direct repeated Apple-Silicon energy-audit environments.

Component	New 17-architecture M4 Pro audit	Retained M5 Pro audit
Device	MacBook Pro Mac16,8	MacBook Pro Mac17,9
Chip	Apple M4 Pro, 12 cores	Apple M5 Pro, 15 cores
Memory	24 GB RAM	24 GB RAM
OS	macOS 26.2 build 25C56	macOS 26.4.1
Python/PyTorch	3.9.6 / 2.8.0	3.12.1 / 2.3.1
torchvision/timm	0.23.0 / 1.0.24	0.18.1 / 1.0.24
Threads/input	1 thread, batch 1	1 thread, batch 1
Power logging	powermetrics, 1 Hz	powermetrics, 1 Hz
Protocol	10 windows/model, 30 s/window	10 windows/model, 20 s/window

TABLE II
Paper-aligned original five-model mean table. EDP is computed from measured energy and latency.

Model	FLOPs (G)	Lat. (ms)	Energy (J)	EDP (J ms)
MobileNetV3-S	0.22	15.14	0.080	1.211
MobileNetV2	0.30	20.07	0.106	2.127
ResNet-18	1.82	20.68	0.110	2.275
Tiny-ViT-5M	1.30	41.33	0.219	9.051
EfficientNet-B0	0.39	55.08	0.292	16.083

result: the same architecture families now have direct powermetrics energy measurements rather than constant-power energy proxies.

The expanded direct audit supports the same qualitative conclusion as the original five-model table. Across 17 architectures, latency explains nearly all measured energy variation ($R^2 = 0.9925$), while FLOPs is much weaker ($R^2 = 0.4008$) and parameter count is weaker still ($R^2 = 0.0977$). The highest-energy group is not simply the highest-FLOP group: ConvNeXt-micro uses 0.448 GFLOPs but consumes slightly more energy than the 3.414-GFLOP residual CNN medium because both run near 6.8 ms per inference. Inverted-residual-wide is the highest-energy architecture despite using less than one GFLOP. These counterexamples show why FLOPs alone is not a reliable deployment-energy proxy for this CPU backend.

C. Backend, Batch, and Input-Size Checks

Table V summarizes the three targeted follow-up audits. The CPU batch-size and input-size checks preserve the central predictor result: latency remains above $R^2 = 0.99$, while FLOPs remains much weaker. The CPU batch-4 ranking is also very close to the main CPU batch-1 ranking (Spearman $\rho = 0.990$ over 17 shared architectures), and the 128×128 CPU ranking remains close over the 15 shared architectures (Spearman $\rho = 0.993$).

MPS/Metal changes the picture. It is much faster and lower-energy for most larger convolutional and residual architectures, but not for the smallest models where dispatch overhead dominates. The MPS energy ranking is still related to the CPU ranking (Spearman $\rho = 0.914$),

but FLOPs becomes the stronger single predictor in this short backend audit ($R^2 = 0.7892$ for FLOPs versus $R^2 = 0.6236$ for latency). This reinforces the main methodological point: direct backend-specific measurement is required, because one runtime’s proxy relationship should not be assumed to transfer unchanged to another runtime.

D. Interpretation for Device Expansion

The original mean table, the earlier five-model M4 audit, the new 17-architecture M4 audit, the three M4 follow-up audits, and the retained M5 summary are intentionally kept separate. The original table supports the earlier paper-aligned comparison; the new M4 audit supplies the strongest expanded direct-energy evidence with raw logs and repeated-window uncertainty; the follow-up audits test whether the main pattern survives backend, batch-size, and input-size changes; and the M5 result is retained as a cross-device audit summary from the prior HPEC artifact. Agreement between them is therefore evidence for robustness of the qualitative pattern, not identity of absolute energy values.

This distinction matters for future device expansion. Apple Silicon CPU-only powermetrics data, Windows CPU data, and Windows GPU data should be reported as separate backend/device conditions. In particular, Windows GPU power logs from tools such as nvidia-smi are not directly equivalent to Apple package-power estimates. They can strengthen the paper if collected with matched model inputs, synchronization around GPU timing, raw power logs, and environment metadata, but they should not be merged into a single undifferentiated energy claim.

V. Discussion

The empirical pattern is consistent across the available Apple-Silicon energy sources: latency is a far stronger proxy for energy than FLOPs. This does not mean latency is a physical cause of energy by itself. Rather, under a fixed runtime and input shape, latency captures many implementation details that FLOPs misses: memory traffic, operator fusion, dispatch overhead, vectorization, cache behavior, and kernel availability.

The largest counterexamples to FLOP-based reasoning now appear in the expanded M4 audit. ConvNeXt-micro

TABLE III
New direct powermetrics audit over 17 architecture variants on Apple M4 Pro. Rows are sorted by measured energy.

Architecture	FLOPs (G)	Lat. (ms)	Energy (J)	95% CI (J)
global_avg_mlp_tiny	0.000	0.021	0.00012	[0.0001,0.0001]
patch_mixer_small	0.042	0.431	0.00284	[0.0025,0.0032]
cnn_tiny_2conv	0.040	0.770	0.00502	[0.0045,0.0055]
patch_mixer_medium	0.101	0.741	0.00534	[0.0044,0.0063]
depthwise_small	0.023	1.289	0.00797	[0.0074,0.0086]
cnn_small_4conv	0.170	1.306	0.00955	[0.0078,0.0113]
grouped_conv_cnn	0.171	1.746	0.01120	[0.0103,0.0121]
depthwise_medium	0.050	1.904	0.01328	[0.0113,0.0153]
cnn_medium_6conv	0.746	2.414	0.01587	[0.0129,0.0189]
attention_gate_cnn	0.526	2.390	0.01663	[0.0145,0.0187]
squeeze_expand_cnn	0.597	3.089	0.01955	[0.0185,0.0206]
residual_cnn_small	1.062	3.434	0.02231	[0.0197,0.0249]
bottleneck_residual	0.724	3.459	0.02252	[0.0203,0.0248]
inverted_residual_small	0.279	3.311	0.02313	[0.0202,0.0261]
residual_cnn_medium	3.414	6.757	0.04259	[0.0387,0.0464]
convnext_micro	0.448	6.832	0.04417	[0.0418,0.0466]
inverted_residual_wide	0.812	6.811	0.04898	[0.0392,0.0588]

TABLE IV
Predictors of measured energy across the main evidence sources.

Dataset and predictor	R^2	Pearson r
Original: latency	0.99999	0.99999
Original: FLOPs	0.00008	-0.009
April M4 audit: latency	0.980	0.990
April M4 audit: FLOPs	0.001	0.034
M4 17-architecture: latency	0.9925	0.9963
M4 17-architecture: FLOPs	0.4008	0.6331
M4 17-architecture: params	0.0977	0.3125
M4 MPS batch 1: latency	0.6236	0.7897
M4 MPS batch 1: FLOPs	0.7892	0.8884
M4 CPU batch 4: latency	0.9914	0.9957
M4 CPU batch 4: FLOPs	0.4397	0.6631
M4 CPU 128 ² : latency	0.9986	0.9993
M4 CPU 128 ² : FLOPs	0.1784	0.4223
Retained M5 audit: latency	0.975	—
Retained M5 audit: FLOPs	0.005	—

has $7.6\times$ fewer FLOPs than residual-CNN-medium but slightly higher measured energy because their latencies are almost the same. Inverted-residual-wide has 0.812 GFLOPs, far below the 3.414 GFLOPs of residual-CNN-medium, but it is the most energy-consuming architecture in the 17-model table. FLOPs alone would therefore make the wrong deployment recommendation for this fixed CPU runtime.

The M4 audits are also valuable because they add uncertainty information absent from the original table. High-power windows are retained rather than removed, and each table entry links back to raw text logs. This makes the evidence more auditable and prevents synthetic data from being confused with physical measurement.

VI. Limitations

The new audit improves the main scope limitation by collecting direct energy for all 17 architecture variants

and adding targeted MPS/Metal, batch-size, and input-size checks. However, the main result remains a local Apple M4 Pro study, and the follow-up audits are shorter than the primary 170-window CPU batch-1 audit. The 128×128 input-size check also covers 15 architectures rather than all 17 because the patch-mixer registry entries are shape-fixed.

The paper still does not evaluate Apple Neural Engine, quantized deployment paths, CUDA, or other vendor-specific GPU inference backends. Those conditions require separate timing synchronization and power logging protocols. The powermetrics sampling interval is also coarse relative to per-inference latency, so energy is attributed at the window level rather than at the level of one forward pass. The local rebuild does not include a raw M5 Pro artifact directory, so the M5 entry is reported only as a retained prior HPEC audit summary. Finally, the residualized WGAN-GP is evaluated against grounded training means and should be interpreted as a supplemental modeling tool, not as proof of physical repeated-trial variance.

VII. Conclusion

On Apple Silicon under CPU-only PyTorch inference, per-inference energy cannot be reduced to FLOPs or parameter count. In the original five-model mean table, latency tracks energy at $R^2 = 0.99999$, while FLOPs explains almost none of the measured variation. The new 17-architecture Apple M4 Pro powermetrics audit strengthens that result with broader direct energy coverage: latency reaches $R^2 = 0.9925$, while FLOPs reaches only $R^2 = 0.4008$. For embedded and edge inference, this supports a practical measurement standard: report latency and auditable power traces under the deployment runtime instead of relying on FLOP counts alone.

TABLE V
Targeted M4 Pro follow-up audits. No measured windows are filtered or winsorized.

Audit	Models	Windows	Lat. R^2	FLOPs R^2	Energy-rank ρ	Largest retained window
CPU batch 1, 224 ²	17	170	0.9925	0.4008	1.000	inverted_residual_wide, 0.0865 J
MPS batch 1, 224 ²	17	85	0.6236	0.7892	0.914	residual_cnn_medium, 0.0273 J
CPU batch 4, 224 ²	17	85	0.9914	0.4397	0.990	residual_cnn_medium, 0.2685 J
CPU batch 1, 128 ²	15	75	0.9986	0.1784	0.993	inverted_residual_wide, 0.0243 J

Artifact Availability

The artifact directory contains the original benchmark CSVs, the measured_energy_powermetrics and measured_energy_powermetrics_m4pro_arch17_cpu directories with raw powermetrics logs, per-window trial CSVs, summary CSVs, environment metadata, measurement scripts, and the supplemental residualized GAN workflow. The follow-up MPS-b1-224, CPU-b4-224, and CPU-b1-128-at-128px directories provide the backend, batch-size, and input-size checks. The retained M5 Pro summary should be paired with its raw artifact directory before final camera-ready archival.

References

- [1] M. Horowitz, “Computing’s energy problem (and what we can do about it),” in IEEE International Solid-State Circuits Conference Digest of Technical Papers, 2014.
- [2] Y.-H. Chen et al., “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” IEEE Journal of Solid-State Circuits, vol. 52, no. 1, pp. 127–138, 2016.
- [3] A. Howard et al., “Searching for MobileNetV3,” in Proc. IEEE/CVF International Conference on Computer Vision, 2019.
- [4] A. G. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision applications,” arXiv:1704.04861, 2017.
- [5] M. Sandler et al., “MobileNetV2: Inverted residuals and linear bottlenecks,” in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018.
- [6] M. Tan and Q. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in Proc. International Conference on Machine Learning, 2019.
- [7] F. N. Iandola et al., “SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size,” arXiv:1602.07360, 2016.
- [8] X. Zhang et al., “ShuffleNet: An extremely efficient convolutional neural network for mobile devices,” in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018.
- [9] R. Schwartz et al., “Green AI,” Communications of the ACM, vol. 63, no. 12, pp. 54–63, 2020.
- [10] L. F. W. Anthony, B. Kanding, and R. Selvan, “Carbontracker: Tracking and predicting the carbon footprint of training deep learning models,” in ICML Workshop on Challenges in Deploying and Monitoring Machine Learning Systems, 2020.
- [11] I. Gulrajani et al., “Improved training of Wasserstein GANs,” in Advances in Neural Information Processing Systems, 2017.